VLSI Design : Course Project

Monsoon 2026, IIIT Hyderabad (Instructor: Prof. Abhishek Srivastava, CVEST)

Release Date: 19-Oct-26, Due date : 1-Dec-26 (18:00 hrs), Demo/Viva date: 3rd & 4th Dec 2026

Instructions:

1. Submit your assignment as a single file in pdf format (Name_RollNo.pdf)
2. Use the given 180 nm technology file for the NGSPICE simulations and $SCN6M_DEEP.09.tech27$ for MAGIC layout
3. Consider lengths of NMOS and PMOS to be equal ($L_n = L_p$), and $V_{DD} = 1.8V$ until stated otherwise
4. Use 'set curplottitle= Your-name-roll-question-number-part' for every plot in your report so that it is printed on the top of each plot
5. Answers should be complete and must be presented in a systematic way with explanation, plots, annotations net-lists and HDL description
6. Utilize moodle platform to discuss and clear your questions. Discussion is highly encouraged.
7. Demo and viva will be conducted on 3rd & 4th Dec 2026. Please book your tickets for 6th Dec onwards.

Part-I: 4-bit Carry Look Ahead Adder (40 Marks)

You are asked to design a 4-bit carry look ahead (CLA) adder as shown in Fig. 1(i). Different modules of the CLA-adder are shown in Fig. 1(ii). Each output sum bit needs to drive an inverter of size W_p/W_n = 20λ/10λ, channel-length = 2λ, where $\lambda = 0.09\mu$m. As shown in Fig. 1(iii), consider that input bits are available before the rising edge of the clock and the output should be computed and present at the next rising edge of the clock. You can choose any logic style (static, dynamic, mix) to implement the circuit.

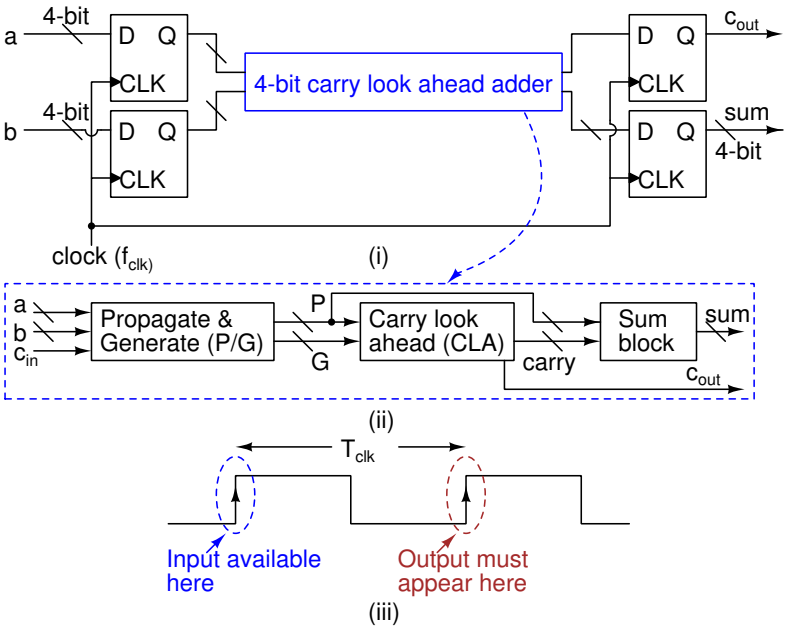


Figure 1

CLA-Adder: If the numbers to be added are $a_3a_2a_1a_0$ and $b_3b_2b_1b_0$, then the propagate (p_i) and generate (g_i) signals for each bit position can be defined as (for $i = 0, 1, 2, 3$):

$$p_i = a_i \oplus b_i$$

$$g_i = a_i \cdot b_i$$

and the carry out (c_{i+1}) of the ith bit position can be written as (assuming input carry c_0):

$$c_{i+1} = g_i + p_i \cdot c_i, \quad i = 0, 1, 2, 3$$

Thus, c_{i+1} can be expressed entirely in terms of the p_i and g_i functions, and the sum bit S_i can be represented as:

$$S_i = p_i \oplus c_i, \quad i = 0, 1, 2, 3$$

1. Briefly discuss your proposed structure for the adder.
2. Give design details (topology and sizing) of each block (D-flip-flop, adder modules).
3. Simulate each block and verify its functionality using NGSPICE.
4. For D-flip-flop, find its setup time, hold time and clock to Q delay from NGSPICE simulations.
5. Layout each block using MAGIC layout editor and previously given technology file. Perform post layout extraction and compare the results with schematic simulation.
6. Integrate different block designed in previous steps and write the netlist for the full circuit shown in figure 1(i). Use NGSPICE and verify the functionality of the circuit. Attach the properly annotated waveforms. Report the worst case delay of your adder and maximum clock speed for which your design operates correctly.
7. Give the floor plan of the layout for the complete circuit. Identify the horizontal and vertical pitches in the regular structures.
8. Make layout of the complete circuit and extract the spice netlist. Repeat the simulations discussed above on the extracted netlist. Give a table comparing the schematic and postlayout simulation results.
9. *Critical Path Delay (Post-Layout):* Identify the worst-case input vector transition (e.g., $A = 1111_2$, $B = 0000_2$ with C_0 toggling $0 \rightarrow 1$) and report the worst-case propagation delay ($t_{pd,\text{max}}$) by using post layout simulations. (*Hint:* When C_0 flips to 1, A and B remain constant, so P_i and G_i stay unchanged. The change in C_0 must evaluate through every level of the Carry Lookahead Generator and Sum logic. The sum vector flips from $S = 1111_2$ to $S = 0000_2$, with $C_4 = 1$. Measure the time from the 50% V_{DD} point of the rising C_0 edge to the 50% V_{DD} point of the slowest transitioning output bit. Check both t_{pLH} and t_{pHL} across $S_0 \ldots S_3$ and C_4; the longest transition time defines the critical path delay ($t_{pd,\text{max}}$). You may use .meas command in ngspice to characterize delays.)
10. Report (Post-Layout) the maximum clock frequency $f_{max} = \frac{1}{t_{pd,max}+t_{setup}+t_{hold}}$ at which your circuit operates reliably. Attach necessary simulation plots.
11. *Power Analysis (Post-Layout):* Measure average dynamic power dissipation at f_{max} and calculate the Power-Delay Product (PDP $= P_{\text{avg}} \times t_{pd,\text{max}}$, $P_{\text{avg}} = \frac{V_{DD}}{T} \int_{T_1}^{T_2} I_{V_{DD}}(t)\, dt$) .

Part-II: SIMD-Reconfigurable 8-bit Adder Design (30 Marks)

Graphical Processing Units (GPUs) often operate on Single Instruction, Multiple Data (SIMD) execution units where a single wide datapath can be dynamically reconfigured into multiple narrower sub-words (e.g., one 8-bit operation vs. two independent 4-bit operations for INT4 inference). *(Ref: John L. Hennessy and David A. Patterson, Computer Architecture: A Quantitative Approach).*

1. Using two instances of your previously designed 4-bit CLA module (CLA_0 and CLA_1), design (netlist and layout) an 8-bit SIMD-Reconfigurable Adder with a control signal `SIMD_MODE` with following *modes of operation:*
   - *8-bit Mode (`SIMD_MODE` $= 0$):* CLA_0 and CLA_1 operate as a single 8-bit adder. Output carry (C_{out}) from CLA_0 propagates into C_{in} of CLA_1.
   - *Dual 4-bit Mode (`SIMD_MODE` $= 1$):* Inter-block carry (C_{out} from CLA_0) is forced to 0, allowing CLA_0 and CLA_1 to execute two independent 4-bit additions in parallel.
2. Implement (netlist and layout) the carry-gating cell using suitable CMOS logic.
3. *Post layout simulations:* Measure and compare the worst-case propagation delay (t_{pd}) in both `SIMD_MODE` $= 0$ and `SIMD_MODE` $= 1$ while driving the specified $W_p/W_n = 20\lambda/10\lambda$ inverter loads.
4. *Using Post layout simulations, calculate the power and delay penalty introduced by the `SIMD_MODE` gating multiplexer compared to a standard 8-bit CLA.* Adding reconfigurability to a fixed data path introduces hardware overhead. The carry-gating cell inserted on the inter-block carry line between CLA_0 and CLA_1 adds extra parasitic capacitance, gate delay, and dynamic switching power. To quantify these architectural penalties, perform a comparative SPICE analysis between a *Baseline Direct 8-bit CLA* (where C_{out} from CLA_0) connects directly to C_{in} of CLA_1 without gating logic) and your *SIMD-Reconfigurable 8-bit CLA*:
   (a) Delay Penalty (Δt_{pd}): Under the worst-case carry propagation vector ($A =$ 0xFF, $B =$ 0x00, with $C_{in,0}$ toggling $0 \rightarrow 1$), measure the propagation delay from $C_{in,0}$ to the most significant sum bit S_7 for both the baseline and reconfigurable adders (`SIMD_MODE` $= 0$). Report the absolute delay penalty (Δt_{pd}) in picoseconds and as a percentage overhead:

$$\text{Delay Penalty (\%)} = \left(\frac{t_{pd,\text{SIMD}} - t_{pd,\text{baseline}}}{t_{pd,\text{baseline}}} \right) \times 100\%$$

(b) Power Penalty (ΔP_{avg}): Simulate the average power consumption (P_{avg}) over 10 clock cycles at maximum operating frequency f_{max} under identical input switching activity. Compare P_{baseline}, $P_{\text{SIMD, mode 0}}$, and $P_{\text{SIMD, mode 1}}$. Report the percentage power penalty introduced by the gating cell in 8-bit mode:

$$\text{Power Penalty (\%)} = \left(\frac{P_{\text{SIMD, mode0}} - P_{\text{baseline}}}{P_{\text{baseline}}} \right) \times 100\%$$

(c) Energy-Delay Product (EDP): Compute and compare the Energy-Delay Product (EDP $= P_{\text{avg}} \times t_{pd}^2$) for both baseline and reconfigurable circuits to evaluate the overall performance trade-off.

Part-III: Verilog & FPGA Implementation (30 Marks)

1. Using Verilog HDL write the behavioural description of 4-bit CLA circuit & SIMD Reconfigurable 8-bit adder. Show the correctness of the functionality using simulations. Clearly describe your test vectors for the proper verification of the designs in all modes. Attach the necessary wave forms and annotate properly.
2. Perform Verilog synthesis and implement both the circuits (Part-I & II) on FPGA. Submit the hardware experimental results by attaching the oscilloscope waveforms. Clearly explain the test scenarios used in hardware validation. Use the same FPGA which is allocated earlier.

Suggested references:

1. Digital Logic and Computer Design by Morris Mano.
2. Computer Architecture and Organization by John P. Hayes.
3. CMOS VLSI Design (fourth edition) by Weste and Harris.
4. Fundamentals of Digital Logic with Verilog Design by Brown and Vranesic.
